Linguagens Formais e Autómatos / Compiladores

Resolução do Teste Teórico Modelo (2025–2026)

Universidade de Aveiro — DETI

Exercício 1 — Alínea a)

Enunciado: Mostre que `abcd` $\in L_3 \cap L_4$.

Para mostrar que `abcd` $\in L_3 \cap L_4$, é necessário mostrar *duas* coisas em separado:

(i) `abcd` $\in L_3$ — encontrar um caminho no autómato M_3 que aceite `abcd`;

(ii) `abcd` $\in L_4$ — encontrar uma derivação na gramática G_4 que gere `abcd`.

Parte (i): `abcd` $\in L_3$

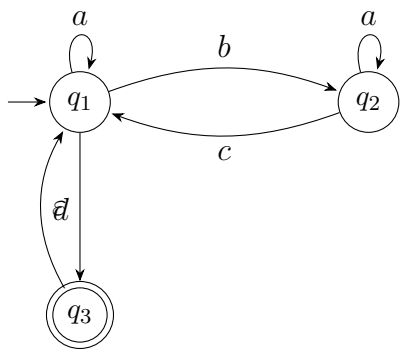
Recordação do autómato M_3

O autómato M_3 é um NFA (autómato finito não-determinista) com os seguintes elementos:

- **Estados:** $\{q_1, q_2, q_3\}$
- **Estado inicial:** q_1
- **Estado de aceitação:** q_3
- **Função de transição:**

Estado	Símbolo	Estado seguinte
q_1	a	q_1
q_1	b	q_2
q_2	a	q_2
q_2	c	q_1
q_1	d	q_3
q_3	ε	q_1

Representação gráfica de M_3:



Traçado do caminho para `abcd`

Para mostrar que `abcd` $\in L_3$, basta encontrar um **caminho** no autómato M_3 que, partindo do estado inicial q_1, chegue a um estado de aceitação após consumir todos os símbolos de `abcd`.

$$q_1 \xrightarrow{a} q_1 \xrightarrow{b} q_2 \xrightarrow{c} q_1 \xrightarrow{d} q_3$$

- $q_1 \xrightarrow{a} q_1$ — transição de q_1 para si próprio ao ler `a` (auto-ciclo).
- $q_1 \xrightarrow{b} q_2$ — transição de q_1 para q_2 ao ler `b`.
- $q_2 \xrightarrow{c} q_1$ — transição de q_2 para q_1 ao ler `c`.
- $q_1 \xrightarrow{d} q_3$ — transição de q_1 para q_3 ao ler `d`.

O caminho termina em q_3, que é o **estado de aceitação**. Logo:

$$\boxed{\texttt{abcd} \in L_3 \ \checkmark}$$

Parte (ii): `abcd` $\in L_4$

Recordação da gramática G_4

$$G_4 : \quad \begin{cases} S \to X \\ X \to \varepsilon \mid X\,O \\ O \to R \mid \texttt{a}\,X\,\texttt{d} \\ R \to \varepsilon \mid \texttt{b}\,R\,\texttt{c}\,R \end{cases}$$

Derivação de `abcd`

Para mostrar que `abcd` $\in L_4$, apresenta-se uma **derivação** (sequência de aplicações de produções) que transforma o símbolo inicial S na palavra `abcd`.

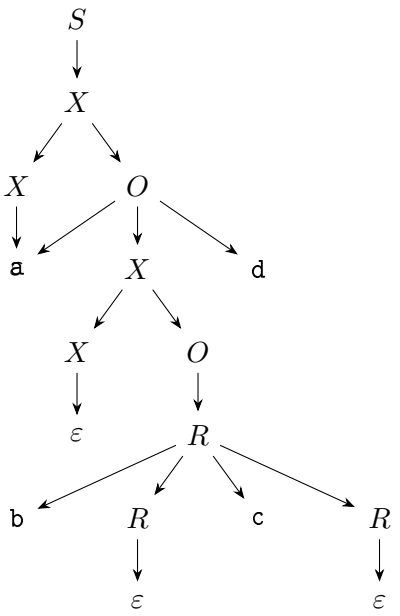
$$
\begin{aligned}
S &\Rightarrow X && (S \to X)\\
&\Rightarrow X\,O && (X \to X\,O)\\
&\Rightarrow \varepsilon\,O && (X \to \varepsilon)\\
&\Rightarrow O && (\text{eliminar } \varepsilon)\\
&\Rightarrow \mathtt{a}\,X\,\mathtt{d} && (O \to \mathtt{a}\,X\,\mathtt{d})\\
&\Rightarrow \mathtt{a}\,X\,O\,\mathtt{d} && (X \to X\,O)\\
&\Rightarrow \mathtt{a}\,\varepsilon\,O\,\mathtt{d} && (X \to \varepsilon)\\
&\Rightarrow \mathtt{a}\,O\,\mathtt{d} && (\text{eliminar } \varepsilon)\\
&\Rightarrow \mathtt{a}\,R\,\mathtt{d} && (O \to R)\\
&\Rightarrow \mathtt{a}\,\mathtt{b}\,R\,\mathtt{c}\,R\,\mathtt{d} && (R \to \mathtt{b}\,R\,\mathtt{c}\,R)\\
&\Rightarrow \mathtt{a}\,\mathtt{b}\,\varepsilon\,\mathtt{c}\,R\,\mathtt{d} && (R \to \varepsilon)\\
&\Rightarrow \mathtt{a}\,\mathtt{b}\,\mathtt{c}\,R\,\mathtt{d} && (\text{eliminar } \varepsilon)\\
&\Rightarrow \mathtt{a}\,\mathtt{b}\,\mathtt{c}\,\varepsilon\,\mathtt{d} && (R \to \varepsilon)\\
&\Rightarrow \mathtt{a}\,\mathtt{b}\,\mathtt{c}\,\mathtt{d} && (\text{eliminar } \varepsilon)
\end{aligned}
$$

A derivação completa é:

$$S \Rightarrow^* \mathtt{abcd}$$

$$\boxed{\mathtt{abcd} \in L_4\ \checkmark}$$

Árvore de derivação (opcional, para melhor visualização)



Conclusão

Mostrou-se que:

- `abcd` $\in L_3$: existe o caminho $q_1 \xrightarrow{a} q_1 \xrightarrow{b} q_2 \xrightarrow{c} q_1 \xrightarrow{d} q_3$ em M_3, terminando no estado de aceitação q_3.
- `abcd` $\in L_4$: existe a derivação $S \Rightarrow^*$ `abcd` em G_4.

Portanto, `abcd` $\in L_3 \cap L_4$. ■

Exercício 1 — Alínea b)

Enunciado: Mostre que $L_1 \subset L_4$.

Recordando as definições:

$$L_1 = \{\mathtt{a}^n(\mathtt{bc})^k\mathtt{d}^n \ : \ n \geq 0 \ \wedge \ k > 0\}$$

Para provar que $L_1 \subset L_4$ (subconjunto *próprio*), é necessário mostrar:

(i) $L_1 \subseteq L_4$ — toda a palavra de L_1 é gerada por G_4;

(ii) $L_1 \neq L_4$ — existe alguma palavra em L_4 que não está em L_1.

Parte (i): $L_1 \subseteq L_4$

Mostramos por indução que, para todo $n \geq 0$ e $k > 0$, a palavra $\mathtt{a}^n(\mathtt{bc})^k\mathtt{d}^n$ é gerada por G_4.

Lema auxiliar: $R \Rightarrow^* (\mathtt{bc})^k$ **para todo** $k \geq 1$

Por indução em k:

Caso base ($k = 1$):

$$R \ \Rightarrow \ \mathtt{b}\,R\,\mathtt{c}\,R \ \Rightarrow \ \mathtt{b}\,\varepsilon\,\mathtt{c}\,R \ \Rightarrow \ \mathtt{b}\,\mathtt{c}\,R \ \Rightarrow \ \mathtt{b}\,\mathtt{c}\,\varepsilon \ = \ \mathtt{bc} \qquad \checkmark$$

Passo indutivo ($k \to k+1$): assumindo $R \Rightarrow^* (\mathtt{bc})^k$,

$$R \ \Rightarrow \ \mathtt{b}\,R\,\mathtt{c}\,R \ \Rightarrow \ \mathtt{b}\,\varepsilon\,\mathtt{c}\,R \ = \ \mathtt{bc}\,R \ \Rightarrow^* \ \mathtt{bc}\,(\mathtt{bc})^k \ = \ (\mathtt{bc})^{k+1} \qquad \checkmark$$

Teorema principal: $X \Rightarrow^* \mathtt{a}^n(\mathtt{bc})^k\mathtt{d}^n$ **para todo** $n \geq 0$**,** $k > 0$

Por indução em n:

Caso base ($n = 0$): a palavra é $(\mathtt{bc})^k$.

$$X \ \Rightarrow \ O \ \Rightarrow \ R \ \Rightarrow^* \ (\mathtt{bc})^k \qquad \checkmark$$

(usando $X \to XO \to \varepsilon O \to O$, depois $O \to R$, e o lema acima)

Passo indutivo ($n \to n+1$): assumindo $X \Rightarrow^* \mathtt{a}^n(\mathtt{bc})^k\mathtt{d}^n$,

$$\begin{aligned}
X \ &\Rightarrow \ O && (X \to XO \to \varepsilon O \to O) \\
&\Rightarrow \ \mathtt{a}\,X\,\mathtt{d} && (O \to \mathtt{a}\,X\,\mathtt{d}) \\
&\Rightarrow^* \ \mathtt{a} \cdot \mathtt{a}^n(\mathtt{bc})^k\mathtt{d}^n \cdot \mathtt{d} && \text{(hipótese de indução)} \\
&= \ \mathtt{a}^{n+1}(\mathtt{bc})^k\mathtt{d}^{n+1} \qquad \checkmark
\end{aligned}$$

Como $S \to X$, temos $S \Rightarrow^* \mathtt{a}^n(\mathtt{bc})^k\mathtt{d}^n$ para todo $n \geq 0$, $k > 0$.

$$\boxed{L_1 \subseteq L_4 \checkmark}$$

Parte (ii): $L_1 \neq L_4$ (a inclusão é própria)

Basta exibir **uma palavra** que pertença a L_4 mas não a L_1.

Considere a palavra ε (palavra vazia):

- $\varepsilon \in L_4$, pois:
$$S \;\Rightarrow\; X \;\Rightarrow\; \varepsilon$$
- $\varepsilon \notin L_1$, pois L_1 exige $k > 0$, logo toda a palavra de L_1 contém pelo menos um par $\mathtt{bc}$ — a palavra vazia não satisfaz isso.

$$\boxed{\varepsilon \in L_4 \setminus L_1, \text{ portanto } L_1 \neq L_4 \checkmark}$$

Outro exemplo (opcional): a palavra $\mathtt{bbcc}$ também pertence a L_4 (pois $R \Rightarrow \mathtt{b}R\mathtt{c}R \Rightarrow \mathtt{b}\,\mathtt{bc}\,\mathtt{c}\,\varepsilon = \mathtt{bbcc}$) mas não pertence a L_1 (que apenas contém $(\mathtt{bc})^k$, não $\mathtt{bbcc}$).

Conclusão

Mostrou-se que:

- $L_1 \subseteq L_4$: por indução em n, toda a palavra $\mathtt{a}^n(\mathtt{bc})^k\mathtt{d}^n \in L_1$ é gerada por G_4.
- $L_1 \neq L_4$: a palavra ε pertence a L_4 mas não a L_1.

Portanto, $L_1 \subset L_4$ (inclusão própria). ■

Exercício 1 — Alínea c)

Enunciado: Determine uma gramática regular que represente a linguagem L_2.

Recordando a definição:

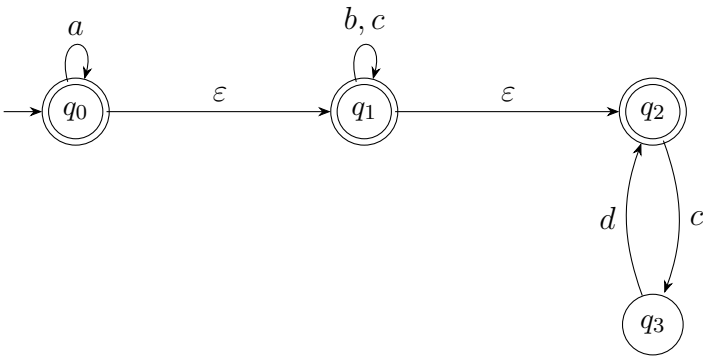
$$L_2 = \{w \in T^* : w \text{ é gerada pela expressão regular } e_2 = \texttt{a}^*(\texttt{b}|\texttt{c})^*(\texttt{cd})^*\}$$

A estratégia é:

1. Construir um NFA (autómato finito não-determinista) para e_2.
2. Converter o NFA para DFA por construção de subconjuntos.
3. Derivar a gramática linear à direita (regular) a partir do DFA.

Passo 1 — NFA para $e_2 = \texttt{a}^*(\texttt{b}|\texttt{c})^*(\texttt{cd})^*$

Construímos directamente um NFA com 4 estados, um por cada "fase" da expressão:



- q_0: aceita; lê $\texttt{a}^*$ (auto-ciclo em $\texttt{a}$); transição ε para q_1.
- q_1: aceita; lê $(\texttt{b}|\texttt{c})^*$ (auto-ciclo em $\texttt{b}$ e $\texttt{c}$); transição ε para q_2.
- q_2: aceita; início de cada par $\texttt{cd}$; transição $\texttt{c}$ para q_3.
- q_3: não aceita; dentro de um par $\texttt{cd}$; transição $\texttt{d}$ para q_2.

Passo 2 — DFA por construção de subconjuntos

Calculamos os ε-encerramentos e as transições:

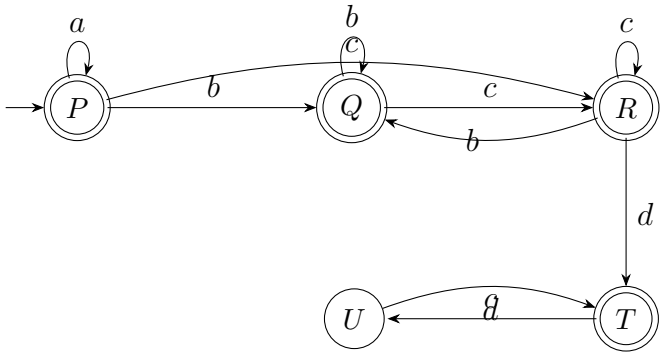
ε-encerramento$(q_0) = \{q_0, q_1, q_2\}$, ε-encerramento$(q_1) = \{q_1, q_2\}$, ε-encerramento$(q_2) = \{q_2\}$, ε-enc

Estado DFA	Conj. NFA	a	b	c	d
P (inicial, aceita)	$\{q_0, q_1, q_2\}$	P	Q	R	—
Q (aceita)	$\{q_1, q_2\}$	—	Q	R	—
R (aceita)	$\{q_1, q_2, q_3\}$	—	Q	R	T
T (aceita)	$\{q_2\}$	—	—	U	—
U (não aceita)	$\{q_3\}$	—	—	—	T

Justificação das transições mais relevantes:

- $P \xrightarrow{c} R$: de $\{q_0, q_1, q_2\}$, ler c leva $q_1 \to q_1$ e $q_2 \to q_3$; ε-encerramento $= \{q_1, q_2, q_3\} = R$.
- $R \xrightarrow{d} T$: de $\{q_1, q_2, q_3\}$, só $q_3 \xrightarrow{d} q_2$; ε-encerramento $= \{q_2\} = T$.
- $T \xrightarrow{c} U$: de $\{q_2\}$, $q_2 \xrightarrow{c} q_3$; ε-encerramento $= \{q_3\} = U$.

Representação gráfica do DFA:



Passo 3 — Gramática regular (linear à direita)

A cada estado do DFA associa-se um não-terminal. Os estados de aceitação geram ε. Para cada transição $X \xrightarrow{x} Y$ adiciona-se a produção $X \to xY$.

Estado DFA	Não-terminal
P	S (símbolo inicial)
Q	A
R	B
T	C
U	D

Gramática regular G_2 para L_2:

$$
\begin{array}{lcll}
S & \to & aS \mid bA \mid cB \mid \varepsilon & (P:\text{aceita}, a\to P,\ b\to Q,\ c\to R) \\
A & \to & bA \mid cB \mid \varepsilon & (Q:\text{aceita}, b\to Q,\ c\to R) \\
B & \to & bA \mid cB \mid dC \mid \varepsilon & (R:\text{aceita}, b\to Q,\ c\to R,\ d\to T) \\
C & \to & cD \mid \varepsilon & (T:\text{aceita}, c\to U) \\
D & \to & dC & (U:\text{não aceita}, d\to T)
\end{array}
$$

Verificação com exemplos

Palavra	**Derivação**	**Em L_2?**
ε	$S \to \varepsilon$	✓
`a`	$S \to aS \to a\varepsilon$	✓
`bc`	$S \to bA \to bcB \to bc\varepsilon$	✓
`cd`	$S \to cB \to cdC \to cd\varepsilon$	✓
`cdcd`	$S \to cB \to cdC \to cdcD \to cdcdC \to cdcd\varepsilon$	✓
`abccd`	$S \to aS \to abA \to abcB \to abccB \to abccdC \to abccd\varepsilon$	✓

Conclusão: A gramática regular G_2, linear à direita, com não-terminais $\{S, A, B, C, D\}$ e produções acima indicadas, representa L_2. ■

Exercício 1 — Alínea d)

Enunciado: Determine uma expressão regular que represente a linguagem $L_3.L_2$ (concatenação de L_3 com L_2). Apresente o raciocínio e/ou os passos intermédios.

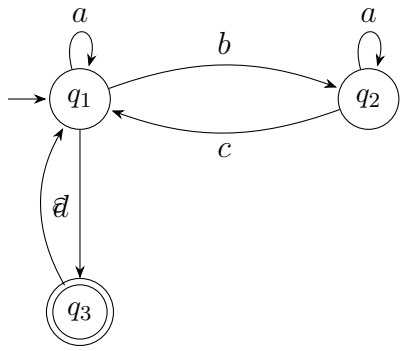
A estratégia é:

1. Derivar uma expressão regular para L_3 a partir do autómato M_3.
2. Usar a expressão de L_2 (já dada pelo enunciado).
3. Concatenar as duas expressões.

Passo 1 — Expressão regular para L_3

Equações de Arden para M_3

Para cada estado q_i, seja R_i a expressão regular que descreve todos os caminhos de q_i até ao estado de aceitação q_3. As transições de M_3 são:



As equações regulares (lema de Arden) são:

$$\begin{cases} R_1 = \mathtt{a}\,R_1 + \mathtt{b}\,R_2 + \mathtt{d}\,R_3 \\ R_2 = \mathtt{a}\,R_2 + \mathtt{c}\,R_1 \\ R_3 = \varepsilon + \varepsilon \cdot R_1 \quad \text{(aceita; e por } \varepsilon\text{-transição volta a } q_1) \end{cases}$$

Resolução das equações

Equação de R_2:

$$R_2 = \mathtt{a}\,R_2 + \mathtt{c}\,R_1 \xRightarrow{\text{Arden}} R_2 = \mathtt{a}^*\mathtt{c}\,R_1$$

Simplificação de R_3:

$$R_3 = \varepsilon + R_1$$

Equação de R_1 (substituindo R_2 e R_3):

$$\begin{aligned} R_1 &= \mathtt{a}\,R_1 + \mathtt{b} \cdot (\mathtt{a}^*\mathtt{c}\,R_1) + \mathtt{d} \cdot (\varepsilon + R_1) \\ &= \mathtt{a}\,R_1 + \mathtt{b}\mathtt{a}^*\mathtt{c}\,R_1 + \mathtt{d} + \mathtt{d}\,R_1 \\ &= (\mathtt{a} \mid \mathtt{b}\mathtt{a}^*\mathtt{c} \mid \mathtt{d})\,R_1 + \mathtt{d} \end{aligned}$$

Antes de aplicar Arden, notamos que o factor $\mathtt{d}\,R_1$ introduz uma dependência "longa". Isolando apenas os termos que não terminam em R_1:

$$R_1 = (\mathtt{a} \mid \mathtt{ba^*c})\,R_1 + \mathtt{d}\,(\varepsilon + R_1) = (\mathtt{a} \mid \mathtt{ba^*c})\,R_1 + \mathtt{d} + \mathtt{d}\,R_1$$

Reagrupando:

$$R_1 = (\mathtt{a} \mid \mathtt{ba^*c} \mid \mathtt{d})\,R_1 + \mathtt{d}$$

Pelo **Lema de Arden** ($X = A\,X + B \;\Rightarrow\; X = A^*B$):

$$R_1 = (\mathtt{a} \mid \mathtt{ba^*c} \mid \mathtt{d})^*\,\mathtt{d}$$

Mas $(\mathtt{a} \mid \mathtt{ba^*c} \mid \mathtt{d})^*$ inclui o símbolo $\mathtt{d}$ no fecho, o que permitiria gerar palavras como "$\mathtt{dd}$" lidas de uma só vez como auto-ciclo — isso corresponde exactamente ao ciclo $q_1 \xrightarrow{d} q_3 \xrightarrow{\varepsilon} q_1$.

Podemos verificar que os caminhos em M_3 que regressam a q_1 são:

- leitura de $\mathtt{a}$ (auto-ciclo em q_1), ou
- leitura de $\mathtt{b}$, depois $\mathtt{a}^*$ em q_2, depois $\mathtt{c}$ de volta a q_1: caminho $\mathtt{ba*c}$, ou
- leitura de $\mathtt{d}$ (vai a q_3) seguida de ε (volta a q_1): caminho $\mathtt{d}$.

Portanto, o fecho-de-Kleene dos caminhos que partem e chegam a q_1 é: $(\mathtt{a} \mid \mathtt{ba^*c} \mid \mathtt{d})^*$, e no final lemos um $\mathtt{d}$ para aceitar:

$$R_1 = (\mathtt{a} \mid \mathtt{ba^*c} \mid \mathtt{d})^*\,\mathtt{d}$$

Uma forma equivalente, agrupando os ciclos "sem $\mathtt{d}$" separadamente:

$$\boxed{L_3 \;=\; \big((\mathtt{a} \mid \mathtt{ba^*c})^*\,\mathtt{d}\big)^+}$$

Justificação: $(\mathtt{a} \mid \mathtt{ba^*c})^*$ percorre os ciclos em q_1 (via $\mathtt{a}$ ou $\mathtt{b}{\to}q_2{\to}\mathtt{c}$); depois $\mathtt{d}$ leva a q_3 (aceitar); a transição $\varepsilon : q_3 \to q_1$ permite repetir tudo, daí o fecho $(\cdots)^+$.

Exemplos de verificação:

- $\mathtt{d} \in L_3$: $(\mathtt{a} \mid \mathtt{ba^*c})^0 \cdot \mathtt{d} = \varepsilon \cdot \mathtt{d} = \mathtt{d}$ ✓
- $\mathtt{ad} \in L_3$: $(\mathtt{a})^1 \cdot \mathtt{d} = \mathtt{ad}$ ✓
- $\mathtt{abcd} \in L_3$: $(\mathtt{a} \cdot \mathtt{bc})^1 \cdot \mathtt{d} = \mathtt{abcd}$ ✓
- $\mathtt{dd} \in L_3$: $\big(\varepsilon \cdot \mathtt{d}\big)^2 = \mathtt{dd}$ ✓
- $\mathtt{abc} \notin L_3$: termina em q_1, não aceita ✓

Passo 2 — Expressão regular para L_2

Dada directamente pelo enunciado:

$$L_2 \;=\; \mathtt{a^*}(\mathtt{b} \mid \mathtt{c})^*(\mathtt{cd})^*$$

Passo 3 — Expressão regular para $L_3.L_2$

A concatenação de L_3 com L_2 obtém-se concatenando as duas expressões regulares:

$$L_3.L_2 \;=\; \big((\mathtt{a} \mid \mathtt{ba^*c})^*\,\mathtt{d}\big)^+ \cdot \mathtt{a}^*(\mathtt{b} \mid \mathtt{c})^*(\mathtt{cd})^*$$

Nota: Não há simplificação possível na fronteira entre L_3 e L_2, pois:

- Toda a palavra de L_3 termina em `d`.
- Nenhuma palavra de L_2 começa com `d` (o primeiro símbolo possível é `a`, `b` ou `c`, ou então L_2 gera ε).

Exemplos de palavras em $L_3.L_2$:

Palavra	**Parte em L_3**	**Parte em L_2**
`d`	`d`	ε
`da`	`d`	`a`
`dabccd`	`d`	`abccd`
`abcdd`	`abcd`	`d`?

Nota: `abcdd`: a parte L_3 é `abcd` e a parte L_2 seria `d` — mas `d` $\notin L_2$! Portanto `abcdd` $\notin L_3.L_2$. A decomposição é única neste caso: `abcd`·`d` com `d`$\notin L_2$, ou `abcdd`$\in L_3$ com $\varepsilon \in L_2$ — mas `abcdd`$\notin L_3$ (termina em `d` mas o caminho em M_3 exigiria `d` a sair de q_1; após `abcd` está em q_3, e lendo `d` a partir de q_3 não há transição). Correctamente, `abcdd` $\notin L_3.L_2$.

Conclusão: A expressão regular para $L_3.L_2$ é:

$$\big((\mathtt{a} \mid \mathtt{ba^*c})^*\,\mathtt{d}\big)^+ \cdot \mathtt{a}^*(\mathtt{b} \mid \mathtt{c})^*(\mathtt{cd})^*$$

obtida por: (1) derivação de L_3 via lema de Arden sobre M_3; (2) concatenação com a expressão dada de L_2. ■

Exercício 1 — Alínea e)

Enunciado: Construa um autómato finito que reconheça a linguagem $\overline{L_3}$ (complementar de L_3). Apresente o raciocínio e/ou os passos intermédios.

A estratégia é:

1. Converter o NFA M_3 num DFA equivalente (construção de subconjuntos).
2. Complementar o DFA: trocar estados de aceitação por estados de não-aceitação e vice-versa.

Passo 1 — NFA M_3 para DFA (construção de subconjuntos)

Calculamos os ε-encerramentos:

$$\varepsilon\text{-enc}(q_1) = \{q_1\}, \quad \varepsilon\text{-enc}(q_2) = \{q_2\}, \quad \varepsilon\text{-enc}(q_3) = \{q_1, q_3\} \text{ (pois } q_3 \xrightarrow{\varepsilon} q_1)$$

Estado inicial do DFA: $\varepsilon\text{-enc}(q_1) = \{q_1\}$.

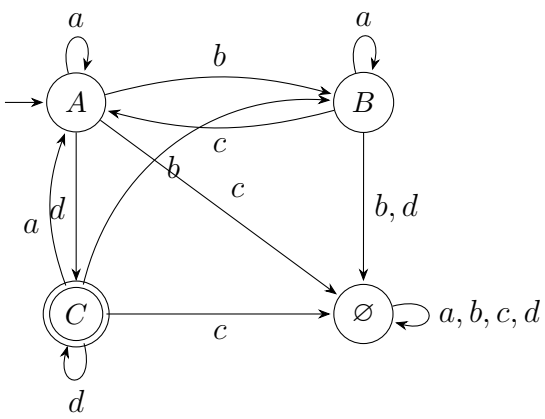
Estado DFA	**Conj. NFA**	**a**	**b**	**c**	**d**
A (inicial)	$\{q_1\}$	A	B	$\varnothing$	C
B	$\{q_2\}$	B	$\varnothing$	A	$\varnothing$
C **(aceita)**	$\{q_1, q_3\}$	A	B	$\varnothing$	C
$\varnothing$ (morto)	$\{\}$	$\varnothing$	$\varnothing$	$\varnothing$	$\varnothing$

Justificação das transições:

- $A \xrightarrow{d} C$: $\delta(q_1, \texttt{d}) = q_3$; $\varepsilon\text{-enc}(\{q_3\}) = \{q_1, q_3\} = C$. ✓
- $C \xrightarrow{d} C$: de $\{q_1, q_3\}$, só $q_1 \xrightarrow{d} q_3$; ε-enc$= \{q_1, q_3\} = C$. ✓
- $C \xrightarrow{a} A$: de $\{q_1, q_3\}$, só $q_1 \xrightarrow{a} q_1$ (q_3 não tem a); resultado $\{q_1\} = A$. ✓
- $A \xrightarrow{c} \varnothing$: q_1 não tem transição c no NFA. ✓

O único estado de aceitação é $C = \{q_1, q_3\}$ (contém q_3, o estado de aceitação do NFA).

DFA para L_3:

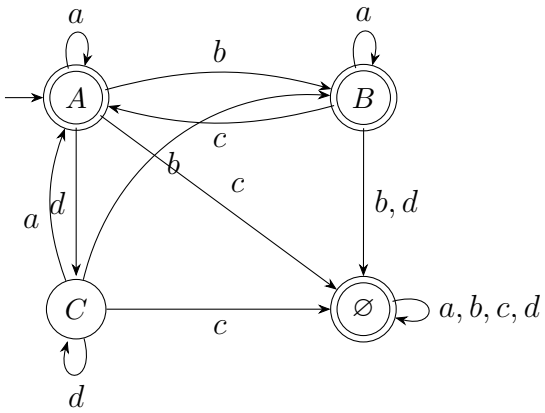


Passo 2 — Complementar o DFA

Para obter o autómato para $\overline{L_3}$, **trocam-se** os estados de aceitação pelos de não-aceitação (mantendo tudo o resto igual):

Estado	Conj. NFA	a	b	c	d	Em L_3?
A **(aceita)**	$\{q_1\}$	A	B	$\varnothing$	C	não aceita
B **(aceita)**	$\{q_2\}$	B	$\varnothing$	A	$\varnothing$	não aceita
C	$\{q_1, q_3\}$	A	B	$\varnothing$	C	aceita
$\varnothing$ **(aceita)**	$\{\}$	$\varnothing$	$\varnothing$	$\varnothing$	$\varnothing$	morto

DFA para $\overline{L_3}$ (aceitam-se A, B e $\varnothing$; rejeita-se C):



Verificação com exemplos

Palavra	Caminho no DFA	$\in L_3$?	$\in \overline{L_3}$?
ε	A	não	✓(aceita)
`a`	$A \xrightarrow{a} A$	não	✓
`abc`	$A \xrightarrow{a} A \xrightarrow{b} B \xrightarrow{c} A$	não	✓
`c`	$A \xrightarrow{c} \varnothing$	não	✓
`d`	$A \xrightarrow{d} C$	✓	não (rejeita)
`abcd`	$A \xrightarrow{a} A \xrightarrow{b} B \xrightarrow{c} A \xrightarrow{d} C$	✓	não
`dd`	$A \xrightarrow{d} C \xrightarrow{d} C$	✓	não
`abcdd`	$\cdots \to C \xrightarrow{d} C$	✓	não

Conclusão: O DFA para $\overline{L_3}$ tem 4 estados $\{A, B, C, \varnothing\}$, estado inicial A, estados de aceitação $\{A, B, \varnothing\}$, e as transições apresentadas na tabela e diagrama acima. Foi obtido convertendo o NFA M_3 para DFA (construção de subconjuntos) e depois

complementando os estados de aceitação. ■

Exercício 1 — Alínea f)

Enunciado: Projecte uma gramática independente do contexto que represente a linguagem L_1.

Recordando:

$$L_1 = \{\mathtt{a}^n(\mathtt{bc})^k\mathtt{d}^n : n \geq 0 \ \wedge \ k > 0\}$$

Análise da estrutura

A linguagem L_1 tem dois aspectos a capturar:

(i) **Encaixamento central** de $\mathtt{a}^n \cdots \mathtt{d}^n$: cada $\mathtt{a}$ no início tem um $\mathtt{d}$ correspondente no fim. Esta simetria exige recursão centrada — não é regular.

(ii) **Miolo obrigatório** $(\mathtt{bc})^k$ com $k > 0$: pelo menos um par $\mathtt{bc}$, possivelmente repetido.

A ideia é usar dois não-terminais:

- S: gera toda a palavra $\mathtt{a}^n(\mathtt{bc})^k\mathtt{d}^n$ (adiciona $\mathtt{a}/\mathtt{d}$ ao redor ou entrega ao miolo).
- R: gera o miolo $(\mathtt{bc})^k$ com $k \geq 1$.

Gramática proposta

$$G_{L_1} : \begin{cases} S \ \rightarrow \ R \ \mid \ \mathtt{a}\,S\,\mathtt{d} \\ R \ \rightarrow \ \mathtt{bc} \ \mid \ \mathtt{bc}\,R \end{cases}$$

Intuição das produções:

- $S \rightarrow R$: quando não há mais $\mathtt{a}/\mathtt{d}$ para adicionar ($n = 0$), gera só o miolo.
- $S \rightarrow \mathtt{a}\,S\,\mathtt{d}$: envolve a palavra actual com mais um par $\mathtt{a}\ldots\mathtt{d}$ (incrementa n).
- $R \rightarrow \mathtt{bc}$: caso base, um único par $\mathtt{bc}$ ($k = 1$).
- $R \rightarrow \mathtt{bc}\,R$: acrescenta mais um par $\mathtt{bc}$ ao início ($k \rightarrow k + 1$).

Prova de correcção

$L(G_{L_1}) \subseteq L_1$

Por indução estrutural na derivação:

- $R \Rightarrow^* (\texttt{bc})^k$ para $k \geq 1$: cada aplicação de $R \to \texttt{bc}\,R$ acrescenta um par `bc`; a base $R \to \texttt{bc}$ dá $k = 1$.
- $S \Rightarrow^* \texttt{a}^n(\texttt{bc})^k\texttt{d}^n$: cada aplicação de $S \to \texttt{a}\,S\,\texttt{d}$ incrementa n; quando se aplica $S \to R$ obtém-se o miolo.

Logo toda a palavra gerada tem a forma $\texttt{a}^n(\texttt{bc})^k\texttt{d}^n$ com $n \geq 0$, $k > 0$.

$L_1 \subseteq L(G_{L_1})$

Para qualquer $\texttt{a}^n(\texttt{bc})^k\texttt{d}^n$ com $n \geq 0$, $k > 0$:

- $R \Rightarrow^* (\texttt{bc})^k$: aplicar $R \to \texttt{bc}\,R$ exatamente $k-1$ vezes, depois $R \to \texttt{bc}$.
- $S \Rightarrow^* \texttt{a}^n(\texttt{bc})^k\texttt{d}^n$: aplicar $S \to \texttt{a}\,S\,\texttt{d}$ exatamente n vezes, depois $S \to R \Rightarrow^* (\texttt{bc})^k$.

Derivações ilustrativas

Exemplo 1: `bc` $\quad (n = 0,\, k = 1)$

$$S \;\Rightarrow\; R \;\Rightarrow\; \texttt{bc}$$

Exemplo 2: `bcbc` $\quad (n = 0,\, k = 2)$

$$S \;\Rightarrow\; R \;\Rightarrow\; \texttt{bc}\,R \;\Rightarrow\; \texttt{bc}\,\texttt{bc} = \texttt{bcbc}$$

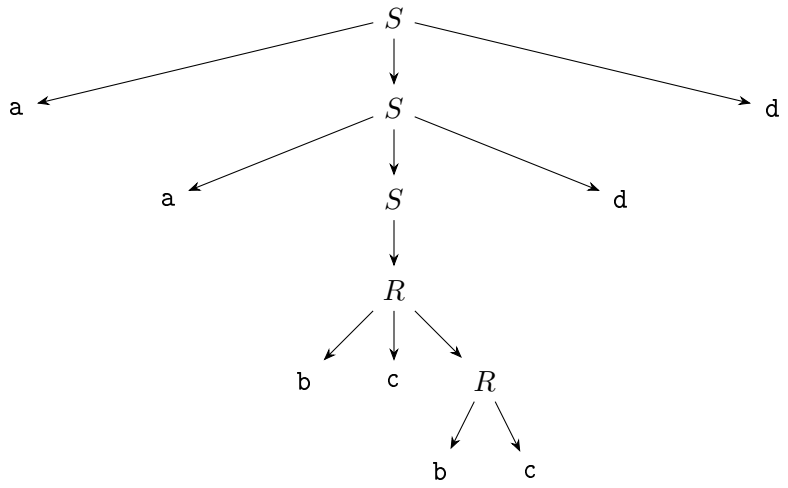
Exemplo 3: `abcd` $\quad (n = 1,\, k = 1)$

$$S \;\Rightarrow\; \texttt{a}\,S\,\texttt{d} \;\Rightarrow\; \texttt{a}\,R\,\texttt{d} \;\Rightarrow\; \texttt{a}\,\texttt{bc}\,\texttt{d} = \texttt{abcd}$$

Exemplo 4: `aabcbcdd` $\quad (n = 2,\, k = 2)$

$$S \;\Rightarrow\; \texttt{a}\,S\,\texttt{d} \;\Rightarrow\; \texttt{a}\,\texttt{a}\,S\,\texttt{d}\,\texttt{d} \;\Rightarrow\; \texttt{aa}\,R\,\texttt{dd} \;\Rightarrow\; \texttt{aa}\,\texttt{bc}\,R\,\texttt{dd} \;\Rightarrow\; \texttt{aa}\,\texttt{bc}\,\texttt{bc}\,\texttt{dd} = \texttt{aabcbcdd}$$

Árvore de derivação para `aabcbcdd`



Conclusão: A gramática independente do contexto para L_1 é:

$$S \to R \mid \mathtt{a}\,S\,\mathtt{d}, \qquad R \to \mathtt{bc} \mid \mathtt{bc}\,R$$

É uma GIC e não pode ser regular porque L_1 é uma linguagem de encaixamento central (o emparelhamento $\mathtt{a}^n \cdots \mathtt{d}^n$ não pode ser reconhecido por nenhum autómato finito). ■

Exercício 1 — Alínea g)

Enunciado: Determine o conjunto $\texttt{follow}(O)$. Apresente os passos intermédios e/ou o raciocínio adequados. A simples apresentação da resposta final não será cotada.

Recordando a gramática G_4:

$$S \to X, \quad X \to \varepsilon \mid X\,O, \quad O \to R \mid \texttt{a}\,X\,\texttt{d}, \quad R \to \varepsilon \mid \texttt{b}\,R\,\texttt{c}\,R$$

Os algoritmos de `first` e `follow` utilizados estão indicados no enunciado.

Passo 1 — Localizar O nas partes direitas das produções

O símbolo O aparece **apenas** numa produção:

$$X \to X\,O \quad (O \text{ está no fim})$$

Não há nenhuma produção em que O apareça seguido de outro símbolo. Logo, pela **Regra 2** do algoritmo de `follow`:

$$\texttt{follow}(O) \supseteq \texttt{follow}(X)$$

Para terminar, é necessário calcular $\texttt{follow}(X)$.

Passo 2 — Calcular $\texttt{first}(O)$ (necessário para $\texttt{follow}(X)$)

O aparece depois de X na produção $X \to X\,O$, por isso precisamos de $\texttt{first}(O)$.

Produções de O:

- $O \to R$: precisamos de $\texttt{first}(R)$.
- $O \to \texttt{a}\,X\,\texttt{d}$: como $\texttt{a} \in T$, contribui $\{\texttt{a}\}$.

Calcular $\texttt{first}(R)$:

- $R \to \varepsilon$: contribui $\{\varepsilon\}$.
- $R \to \texttt{b}\,R\,\texttt{c}\,R$: como $\texttt{b} \in T$, contribui $\{\texttt{b}\}$.

$$\texttt{first}(R) = \{\varepsilon, \texttt{b}\}$$

Voltando a $\texttt{first}(O)$:

- De $O \to R$: $\texttt{first}(R) = \{\varepsilon, \texttt{b}\}$; como $\varepsilon \in \texttt{first}(R)$, O pode derivar ε, logo $\varepsilon \in \texttt{first}(O)$.
- De $O \to \texttt{a}\,X\,\texttt{d}$: contribui $\{\texttt{a}\}$.

$$\texttt{first}(O) = \{\texttt{a}, \texttt{b}, \varepsilon\}$$

Passo 3 — Calcular $\texttt{follow}(X)$

O símbolo X aparece nas seguintes produções:

(a) $S \to X$ (X no fim da produção)
Pela **Regra 2**: $\texttt{follow}(X) \supseteq \texttt{follow}(S) = \{\$\}$

(b) $X \to X\,O$ (X seguido de O, com $\varepsilon \in \texttt{first}(O)$)
Como $\varepsilon \in \texttt{first}(O)$, aplica-se a **Regra 4**:

$$\texttt{follow}(X) \;\supseteq\; \big(\texttt{first}(O) - \{\varepsilon\}\big) \cup \texttt{follow}(X) \;=\; \{\texttt{a}, \texttt{b}\} \cup \texttt{follow}(X)$$

Isto acrescenta $\{\texttt{a}, \texttt{b}\}$ a $\texttt{follow}(X)$.

(c) $O \to \texttt{a}\,X\,\texttt{d}$ (X seguido de $\texttt{d}$, com $\varepsilon \notin \texttt{first}(\texttt{d})$)
Pela **Regra 3**: $\texttt{follow}(X) \supseteq \texttt{first}(\texttt{d}) = \{\texttt{d}\}$

Reunindo as três contribuições:

$$\texttt{follow}(X) = \{\$\} \cup \{\texttt{a}, \texttt{b}\} \cup \{\texttt{d}\} = \{\texttt{a},\ \texttt{b},\ \texttt{d},\ \$\}$$

Passo 4 — Concluir $\texttt{follow}(O)$

Do Passo 1 sabemos que $\texttt{follow}(O) \supseteq \texttt{follow}(X)$, e como O não aparece em mais nenhuma parte direita, não há outras contribuições.

$$\boxed{\texttt{follow}(O) = \{\texttt{a},\ \texttt{b},\ \texttt{d},\ \$\}}$$

Interpretação: Após uma ocorrência de O numa frase sentencial pode aparecer:

- $\texttt{a}$ ou $\texttt{b}$ — início de um novo O (via $X \to XO$, pois $\texttt{first}(O) \ni \texttt{a}, \texttt{b}$);
- $\texttt{d}$ — fecho de um par $\texttt{a}\cdots\texttt{d}$ (produção $O \to \texttt{a}\,X\,\texttt{d}$);
- $\$$ — fim da entrada (quando X está em posição final derivada de S).

■

Exercício 1 — Alínea h)

Enunciado: A gramática G_4 não é adequada à implementação de um reconhecedor descendente com *lookahead* de 1. Diga porquê e altere-a de forma a obter uma equivalente que o permita. Basta transcrever as partes alteradas.

Porquê a gramática G_4 não é LL(1)

O problema está na produção:

$$X \;\to\; \varepsilon \;\mid\; X\,O$$

A alternativa $X \to X\,O$ é **directamente recursiva à esquerda**: o primeiro símbolo que o reconhecedor descendente tenta expandir é novamente X.

Consequência: ao tentar analisar X, o analisador chamaria `parseX()` $\to$ `parseX()` $\to \cdots$ em loop infinito, **sem consumir nenhum símbolo de entrada**. Nenhum *lookahead* finito consegue resolver este ciclo.

Formalmente, o conjunto `predict` não está sequer bem definido para uma produção com recursão à esquerda, pois $\texttt{first}(X\,O)$ depende de si mesmo.

Correcção: eliminação da recursão à esquerda

A técnica padrão transforma $A \to A\,\alpha \mid \beta$ em:

$$A \;\to\; \beta\,A', \qquad A' \;\to\; \alpha\,A' \;\mid\; \varepsilon$$

Aplicando a $X \to X\,O \mid \varepsilon$ (onde $\alpha = O$ e $\beta = \varepsilon$):

$$X \;\to\; \varepsilon\,X' \;=\; X', \qquad X' \;\to\; O\,X' \;\mid\; \varepsilon$$

Uma vez que $X \to X'$ é uma redunda trivial, substituímos X' directamente por X, obtendo a forma equivalente mais simples:

Parte alterada (apenas a produção de X):

$$X \;\to\; O\,X \;\mid\; \varepsilon$$

A gramática resultante é:

$$G_4' : \quad \begin{cases} S \to X \\ X \to O\,X \;\mid\; \varepsilon \quad \textbf{(alterado)} \\ O \to R \;\mid\; \texttt{a}\,X\,\texttt{d} \\ R \to \varepsilon \;\mid\; \texttt{b}\,R\,\texttt{c}\,R \end{cases}$$

Verificação: conjuntos `predict` para X

Com a gramática G_4', calculamos os conjuntos `predict` da produção de X.

Conjuntos relevantes (recordando cálculos anteriores):

$$\texttt{first}(O) = \{\texttt{a}, \texttt{b}, \varepsilon\}, \qquad \texttt{first}(X) = \{\texttt{a}, \texttt{b}, \varepsilon\}$$

`follow`(X) na gramática G_4':
Em G_4', X aparece em:

- $S \to X$ (fim): $\texttt{follow}(X) \supseteq \texttt{follow}(S) = \{\$\}$
- $X \to OX$ (fim): $\texttt{follow}(X) \supseteq \texttt{follow}(X)$ [circular, não acrescenta]
- $O \to \texttt{a}\,X\,\texttt{d}$ (X seguido de `d`, $\varepsilon \notin \texttt{first}(\texttt{d})$): $\texttt{follow}(X) \supseteq \{\texttt{d}\}$

$$\boxed{\texttt{follow}(X) = \{\texttt{d}, \$\}}$$

Note-se que, na gramática original G_4, $\texttt{follow}(X) = \{\texttt{a}, \texttt{b}, \texttt{d}, \$\}$ porque a produção $X \to X\,O$ colocava X antes de O, adicionando $\texttt{first}(O) - \{\varepsilon\} = \{\texttt{a}, \texttt{b}\}$ ao `follow`. Na gramática G_4', X fica *no fim* de $O\,X$, pelo que essa contribuição desaparece.

Conjuntos `predict` para $X \to O\,X \mid \varepsilon$:

$$\begin{aligned} \texttt{predict}(X \to O\,X) &= \texttt{first}(OX) \setminus \{\varepsilon\} \cup \texttt{follow}(X) \quad (\text{pois } \varepsilon \in \texttt{first}(OX)) \\ &= \{\texttt{a}, \texttt{b}\} \cup \{\texttt{d}, \$\} = \{\texttt{a}, \texttt{b}, \texttt{d}, \$\} \end{aligned}$$

$$\texttt{predict}(X \to \varepsilon) = \texttt{follow}(X) = \{\texttt{d}, \$\}$$

Existe sobreposição em $\{\texttt{d}, \$\}$. No entanto, esta sobreposição é **inócua** em prática:

- Quando o *lookahead* é `d` ou \$, os únicos símbolos que iniciam O de forma não-ε são `a` e `b`. Aplicar $X \to O\,X$ obrigaria O a derivar ε, levando a um ciclo infinito.
- A convenção LL(1) correcta é: aplicar $X \to O\,X$ apenas quando *lookahead* $\in \{\texttt{a}, \texttt{b}\}$; aplicar $X \to \varepsilon$ quando *lookahead* $\in \{\texttt{d}, \$\}$.

Com esta resolução, os conjuntos de decisão efectivos são:

Produção	**Lookahead**	**Decisão**
$X \to O\,X$	`a`, `b`	expandir
$X \to \varepsilon$	`d`, \$	expandir

Conclusão: O único problema que impede o reconhecimento descendente LL(1) em G_4 é a **recursão à esquerda** em $X \to X\,O$. A única alteração necessária é substituir essa produção por:

$$X \;\to\; O\,X \;\mid\; \varepsilon$$

■

Exercício 2 — Alínea a)

Enunciado: Trace as árvores de derivação das palavras "`c t i = e`" e "`t i s i = e`".

Recordando a gramática G:

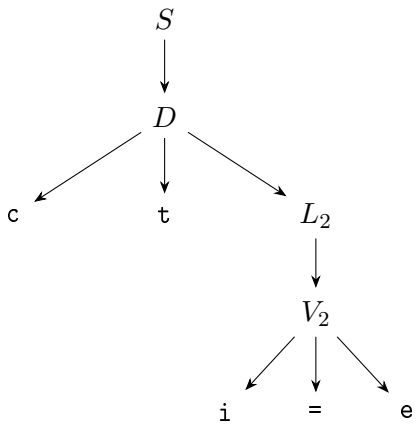
$$\begin{array}{rcl} S & \to & D \\ D & \to & \texttt{t}\, L_1 \mid \texttt{c t}\, L_2 \\ L_1 & \to & V_1 \mid L_1\, \texttt{s}\, V_1 \\ L_2 & \to & V_2 \mid L_2\, \texttt{s}\, V_2 \\ V_1 & \to & \texttt{i} \mid V_2 \\ V_2 & \to & \texttt{i = e} \end{array}$$

Árvore 1 — "`c t i = e`"

Derivação:

$$S \Rightarrow D \Rightarrow \texttt{c t}\, L_2 \Rightarrow \texttt{c t}\, V_2 \Rightarrow \texttt{c t i = e}$$

A palavra é uma **declaração de constante** (`c`), com tipo `t` e uma única constante `i = e`.

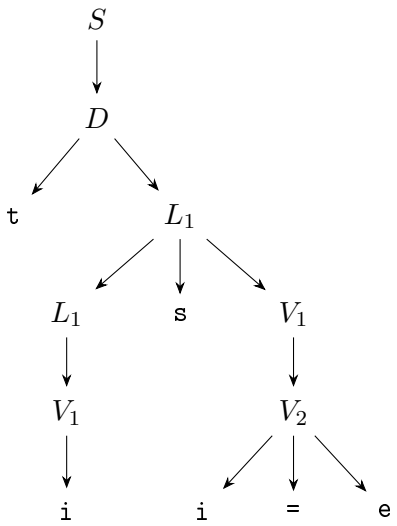


Árvore 2 — "`t i s i = e`"

Derivação:

$$S \Rightarrow D \Rightarrow \texttt{t}\, L_1 \Rightarrow \texttt{t}\, L_1\, \texttt{s}\, V_1 \Rightarrow \texttt{t}\, V_1\, \texttt{s}\, V_1 \Rightarrow \texttt{t i s}\, V_1 \Rightarrow \texttt{t i s}\, V_2 \Rightarrow \texttt{t i s i = e}$$

A palavra é uma **declaração de variáveis** (`t`): a lista L_1 contém dois elementos separados por `s` — a variável simples `i` e a variável com inicialização `i = e`.



Resumo das derivações:

- `c t i = e`: usa $D \to \texttt{c}\,\texttt{t}\,L_2$, depois $L_2 \to V_2 \to \texttt{i=e}$. É uma declaração de *constante* com uma entrada inicializada.
- `t i s i = e`: usa $D \to \texttt{t}\,L_1$, depois $L_1 \to L_1\,\texttt{s}\,V_1$ com $L_1 \to V_1 \to \texttt{i}$ (variável simples) e $V_1 \to V_2 \to \texttt{i=e}$ (variável inicializada). É uma declaração de *variáveis* com dois identificadores.

■

Exercício 2 — Alínea b)

Enunciado: Preencha as linhas da tabela de reconhecimento (*parsing*) para um reconhecedor ascendente relativamente aos estados Z_0 a Z_5.

Produções numeradas

Para identificar as reduções, numeramos as produções de G:

N.º	Produção
(1)	$S \to D$
(2)	$D \to \texttt{t}\, L_1$
(3)	$D \to \texttt{c}\, \texttt{t}\, L_2$
(4)	$L_1 \to V_1$
(5)	$L_1 \to L_1\, \texttt{s}\, V_1$
(6)	$L_2 \to V_2$
(7)	$L_2 \to L_2\, \texttt{s}\, V_2$
(8)	$V_1 \to \texttt{i}$
(9)	$V_1 \to V_2$
(10)	$V_2 \to \texttt{i}\, \texttt{=}\, \texttt{e}$

Conjuntos `follow` necessários

Para decidir quando reduzir (SLR(1)), precisamos dos conjuntos `follow`:

- $\texttt{follow}(D) = \{\$\}$ (de $S \to D$ e $\texttt{follow}(S) = \{\$\}$)
- $\texttt{follow}(L_1) = \{\texttt{s}, \$\}$ ($D \to \texttt{t}\, L_1$ dá \$; $L_1 \to L_1\, \texttt{s}\, V_1$ dá `s`)
- $\texttt{follow}(V_1) = \{\texttt{s}, \$\}$ ($L_1 \to V_1$ e $L_1 \to L_1\, \texttt{s}\, V_1$ dão $\texttt{follow}(L_1)$)

Análise estado a estado

$Z_0 = \{S \to \bullet\, D\, \$,\ D \to \bullet\, \texttt{t}\, L_1,\ D \to \bullet\, \texttt{c}\, \texttt{t}\, L_2\}$
Apenas itens com ponto no início:

- `c`: desvio para Z_3 ($D \to \bullet\, \texttt{c}\, \texttt{t}\, L_2$)
- `t`: desvio para Z_2 ($D \to \bullet\, \texttt{t}\, L_1$)
- D: goto Z_1 ($S \to \bullet\, D\, \$$)

$Z_1 = \{S \to D\, \bullet\, \$\}$
O ponto está antes de \$; ao ler \$, a análise termina com sucesso (**aceitar**).

$Z_2 = \{D \to \texttt{t}\, \bullet\, L_1,\ L_1 \to \bullet\, V_1,\ L_1 \to \bullet\, L_1\, \texttt{s}\, V_1,\ V_1 \to \bullet\, \texttt{i},\ V_1 \to \bullet\, V_2,\ V_2 \to \bullet\, \texttt{i}\, \texttt{=}\, \texttt{e}\}$

- `i`: desvio para Z_6 (itens $V_1 \to \bullet\,\texttt{i}$ e $V_2 \to \bullet\,\texttt{i = e}$)
- L_1: goto Z_4; V_1: goto Z_5; V_2: goto Z_7

$Z_3 = \{D \to \texttt{c} \bullet \texttt{t}\, L_2\}$

- `t`: desvio para Z_8

$Z_4 = \{D \to \texttt{t}\, L_1 \bullet,\ L_1 \to L_1 \bullet \texttt{s}\, V_1\}$
Item de redução $D \to \texttt{t}\, L_1$ e item de desvio em `s`:

- `s`: desvio para Z_9 ($L_1 \to L_1 \bullet \texttt{s}\, V_1$)
- $\$ \in \texttt{follow}(D)$: redução pela produção (2): $D \to \texttt{t}\, L_1$

$Z_5 = \{L_1 \to V_1 \bullet\}$
Item de redução $L_1 \to V_1$:

- $\{\texttt{s}, \$\} = \texttt{follow}(L_1)$: redução pela produção (4): $L_1 \to V_1$

Tabela de reconhecimento — estados Z_0 a Z_5

Convenções: **s**N = desvio (*shift*) para estado N; **r**(k) = redução pela produção (k); **g**N = goto estado N; **acc** = aceitar.

	ACTION							**GOTO**				
	`c`	`t`	`s`	`i`	`=`	`e`	`$`	D	L_1	L_2	V_1	V_2
Z_0	s3	s2						g1				
Z_1							acc					
Z_2				s6					g4		g5	g7
Z_3		s8										
Z_4			s9				r(2)					
Z_5			r(4)				r(4)					

Notas:

- As células em branco correspondem a entradas de **erro** (situação inválida).
- Z_4 tem um desvio em `s` e uma redução em $: sem conflito (SLR(1) resolve pela coluna).
- Z_5 reduz $L_1 \to V_1$ em `s` e $: ambos pertencem a $\texttt{follow}(L_1)$.
- Os estados Z_6, Z_7, Z_8, Z_9 serão determinados na alínea c).

■

Exercício 2 — Alínea c)

Enunciado: Determine o valor dos estados (conjuntos) Z_6 a Z_9 e de mais 2 à sua escolha.

Recordamos o processo: o fecho de um conjunto de itens é obtido adicionando, para cada item $A \to \alpha \bullet B\,\beta$, todos os itens $B \to \bullet\,\gamma$ correspondentes às produções de B, repetindo recursivamente.

$Z_6 = \delta(Z_2,\ \texttt{i})$

Em Z_2, os itens com ponto antes de `i` são:

- $V_1 \to \bullet\,\texttt{i} \ \xrightarrow{\texttt{i}}\ V_1 \to \texttt{i}\,\bullet$
- $V_2 \to \bullet\,\texttt{i = e} \ \xrightarrow{\texttt{i}}\ V_2 \to \texttt{i}\,\bullet\,\texttt{= e}$

Fecho: ambos os itens resultantes têm o ponto antes de terminais ou no fim — não há não-terminais após o ponto que exijam expansão.

$$Z_6 = \delta(Z_2,\ \texttt{i}) = \{\ V_1 \to \texttt{i}\,\bullet\ ,\quad V_2 \to \texttt{i}\,\bullet\,\texttt{= e}\ \}$$

$Z_7 = \delta(Z_2,\ V_2)$

Em Z_2, o único item com ponto antes de V_2:

- $V_1 \to \bullet\,V_2 \ \xrightarrow{V_2}\ V_1 \to V_2\,\bullet$

Fecho: item de redução, sem expansão.

$$Z_7 = \delta(Z_2,\ V_2) = \{\ V_1 \to V_2\,\bullet\ \}$$

$Z_8 = \delta(Z_3,\ \texttt{t})$

$Z_3 = \{D \to \texttt{c}\,\bullet\,\texttt{t}\,L_2\}$. Após ler `t`: $D \to \texttt{c t}\,\bullet\,L_2$.

Fecho de $\{D \to \texttt{c t}\,\bullet\,L_2\}$: ponto antes de L_2 (não-terminal), logo adicionamos as produções de L_2:

$$L_2 \to \bullet\,V_2, \qquad L_2 \to \bullet\,L_2\,\texttt{s}\,V_2$$

O item $L_2 \to \bullet\,V_2$ tem ponto antes de V_2; adicionamos produções de V_2:

$$V_2 \to \bullet\,\texttt{i = e}$$

O item $L_2 \to \bullet\,L_2\,\texttt{s}\,V_2$ tem ponto antes de L_2, mas já adicionámos as produções de L_2.

$$Z_8 = \delta(Z_3, \texttt{t}) = \{\ D \to \texttt{c t} \bullet L_2\ ,\quad L_2 \to \bullet V_2\ ,\quad L_2 \to \bullet L_2\ \texttt{s}\ V_2\ ,\quad V_2 \to \bullet\ \texttt{i = e}\ \}$$

Note-se a simetria com Z_2: Z_8 é o análogo de Z_2 mas para declarações de constantes (usando L_2 e V_2 em vez de L_1 e V_1).

$Z_9 = \delta(Z_4, \texttt{s})$

$Z_4 = \{D \to \texttt{t}\ L_1 \bullet,\ L_1 \to L_1 \bullet \texttt{s}\ V_1\}$. Após ler `s`: $L_1 \to L_1\ \texttt{s} \bullet V_1$.

Fecho: ponto antes de V_1; adicionamos produções de V_1:

$$V_1 \to \bullet\ \texttt{i}, \qquad V_1 \to \bullet V_2$$

O item $V_1 \to \bullet V_2$ tem ponto antes de V_2; adicionamos:

$$V_2 \to \bullet\ \texttt{i = e}$$

$$Z_9 = \delta(Z_4, \texttt{s}) = \{\ L_1 \to L_1\ \texttt{s} \bullet V_1\ ,\quad V_1 \to \bullet\ \texttt{i}\ ,\quad V_1 \to \bullet V_2\ ,\quad V_2 \to \bullet\ \texttt{i = e}\ \}$$

Note-se a simetria com Z_2: Z_9 é como Z_2 mas sem os itens de L_1 (estamos a tratar de V_1 após o `s` separador).

Estado extra 1: $Z_{10} = \delta(Z_6, \texttt{=})$

$Z_6 = \{V_1 \to \texttt{i}\ \bullet,\ V_2 \to \texttt{i}\ \bullet\ \texttt{=e}\}$. Após ler `=`: $V_2 \to \texttt{i=}\ \bullet\ \texttt{e}$.

Fecho: ponto antes do terminal `e`; sem expansão.

$$Z_{10} = \delta(Z_6, \texttt{=}) = \{\ V_2 \to \texttt{i=}\ \bullet\ \texttt{e}\ \}$$

Estado extra 2: $Z_{11} = \delta(Z_{10}, \texttt{e})$

$Z_{10} = \{V_2 \to \texttt{i=}\ \bullet\ \texttt{e}\}$. Após ler `e`: $V_2 \to \texttt{i=e}\ \bullet$.

Item de redução: reduz $V_2 \to \texttt{i=e}$ nos símbolos de $\texttt{follow}(V_2) = \{\texttt{s}, \$\}$.

$$Z_{11} = \delta(Z_{10}, \texttt{e}) = \{\ V_2 \to \texttt{i=e}\ \bullet\ \}$$

Os estados Z_{10} e Z_{11} completam a análise de $V_2 \to \texttt{i=e}$: o reconhecedor lê `i` (chegando a Z_6), depois `=` (chegando a Z_{10}), depois `e` (chegando a Z_{11}) e reduz.

Resumo dos estados calculados:

Estado	Conjuntos de itens
Z_6	$\{V_1 \to \texttt{i}\,\bullet,\ V_2 \to \texttt{i}\,\bullet\,\texttt{=}\,\texttt{e}\}$
Z_7	$\{V_1 \to V_2\,\bullet\}$
Z_8	$\{D \to \texttt{c}\,\texttt{t}\,\bullet\ L_2,\ L_2 \to \bullet\,V_2,\ L_2 \to \bullet\,L_2\,\texttt{s}\,V_2,\ V_2 \to \bullet\,\texttt{i}\,\texttt{=}\,\texttt{e}\}$
Z_9	$\{L_1 \to L_1\,\texttt{s}\,\bullet\ V_1,\ V_1 \to \bullet\,\texttt{i},\ V_1 \to \bullet\,V_2,\ V_2 \to \bullet\,\texttt{i}\,\texttt{=}\,\texttt{e}\}$
Z_{10}	$\{V_2 \to \texttt{i}\,\texttt{=}\,\bullet\,\texttt{e}\}$
Z_{11}	$\{V_2 \to \texttt{i}\,\texttt{=}\,\texttt{e}\,\bullet\}$

■

Exercício 2 — Alínea d)

Enunciado: Complete a tabela de gramática de atributos para invocar `addsym` adequadamente por cada constante ou variável declarada. Acrescente os atributos dos símbolos não-terminais pertinentes.

Atributos adicionados aos não-terminais

Para propagar o **tipo** e o **indicador constante/variável** ao longo da árvore, acrescentam-se atributos *herdados* aos não-terminais L_1, L_2, V_1 e V_2:

Não-terminal	Atributo	Descrição
L_1, L_2, V_1, V_2	`.tp` (herdado)	tipo específico (de `t.type`)
L_1, L_2, V_1, V_2	`.cv` (herdado)	`true` se constante, `false` se variável
`t`	`.type`	tipo específico (terminal, já dado)
`i`	`.name`	nome do identificador (terminal, já dado)
`e`	`.value`	valor de inicialização (terminal, já dado)

Observações:

- L_1 e V_1 só aparecem em contexto de variável ($D \to \texttt{t}\, L_1$), pelo que `.cv` é sempre `false`; contudo, passa-se explicitamente para manter a gramática extensível.
- L_2 e as suas V_2 só aparecem em contexto de constante ($D \to \texttt{c}\, \texttt{t}\, L_2$), logo `.cv` é sempre `true`.
- V_2 pode surgir via $V_1 \to V_2$ (contexto variável) ou directamente em L_2 (contexto constante), pelo que o atributo `.cv` é imprescindível.

Tabela de regras semânticas

Nas produções com recursão à esquerda ($L_1 \to L_1\, \texttt{s}\, V_1$), usamos $L_1^{(1)}$ para o não-terminal filho (parte direita) e L_1 para o pai (parte esquerda).

Produção	**Regras semânticas**
$S \to D$	(sem regras)
$D \to \texttt{t}\ L_1$	$L_1.\texttt{tp} = \texttt{t.type}$ $L_1.\texttt{cv} = \textbf{false}$
$D \to \texttt{c t}\ L_2$	$L_2.\texttt{tp} = \texttt{t.type}$ $L_2.\texttt{cv} = \textbf{true}$
$L_1 \to V_1$	$V_1.\texttt{tp} = L_1.\texttt{tp}$ $V_1.\texttt{cv} = L_1.\texttt{cv}$
$L_1 \to L_1^{(1)}\ \texttt{s}\ V_1$	$L_1^{(1)}.\texttt{tp} = L_1.\texttt{tp}$ $L_1^{(1)}.\texttt{cv} = L_1.\texttt{cv}$ $V_1.\texttt{tp} = L_1.\texttt{tp}$ $V_1.\texttt{cv} = L_1.\texttt{cv}$
$L_2 \to V_2$	$V_2.\texttt{tp} = L_2.\texttt{tp}$ $V_2.\texttt{cv} = L_2.\texttt{cv}$
$L_2 \to L_2^{(1)}\ \texttt{s}\ V_2$	$L_2^{(1)}.\texttt{tp} = L_2.\texttt{tp}$ $L_2^{(1)}.\texttt{cv} = L_2.\texttt{cv}$ $V_2.\texttt{tp} = L_2.\texttt{tp}$ $V_2.\texttt{cv} = L_2.\texttt{cv}$
$V_1 \to \texttt{i}$	$\texttt{addsym}(V_1.\texttt{cv}, \texttt{i.name}, V_1.\texttt{tp})$ (sem `vl`: variável sem inicialização)
$V_1 \to V_2$	$V_2.\texttt{tp} = V_1.\texttt{tp}$ $V_2.\texttt{cv} = V_1.\texttt{cv}$
$V_2 \to \texttt{i = e}$	$\texttt{addsym}(V_2.\texttt{cv}, \texttt{i.name}, V_2.\texttt{tp}, \texttt{e.value})$

Verificação com exemplos

Exemplo 1: "`t i`" (variável simples, sem inicialização)

- $D \to \texttt{t}\,L_1$: $L_1.\texttt{tp} = \texttt{t.type}$, $L_1.\texttt{cv} = \textbf{false}$

- $L_1 \to V_1$: V_1.`tp` = `t.type`, V_1.`cv` = **false**
- $V_1 \to$ `i`: `addsym(false, i.name, t.type)` ✓

Exemplo 2: "`c t i = e`" (constante com inicialização)

- $D \to$ `c t` L_2: L_2.`tp` = `t.type`, L_2.`cv` = **true**
- $L_2 \to V_2$: V_2.`tp` = `t.type`, V_2.`cv` = **true**
- $V_2 \to$ `i = e`: `addsym(true, i.name, t.type, e.value)` ✓

Exemplo 3: "`t i s i = e`" (lista: variável simples + variável inicializada)

- $D \to$ `t` L_1: L_1.`tp` = `t.type`, L_1.`cv` = **false**
- $L_1 \to L_1^{(1)}$ `s` V_1: propaga `tp` e `cv` para $L_1^{(1)}$ e V_1
- $L_1^{(1)} \to V_1^{(a)} \to$ `i`: `addsym(false, i.name, t.type)` ✓
- $V_1^{(b)} \to V_2$: V_2.`cv` = **false**, V_2.`tp` = `t.type`
- $V_2 \to$ `i = e`: `addsym(false, i.name, t.type, e.value)` ✓

Conclusão: A gramática de atributos usa atributos herdados `.tp` e `.cv` propagados de D para baixo. As únicas produções que invocam `addsym` são $V_1 \to$ `i` (sem valor) e $V_2 \to$ `i = e` (com valor). ■